Polytech' Nice Deep Learning course 2025

Deep Learning

By Andrea Binetruy Bilon and Alberto García de la Torre

Experiments and Model Evolution

To begin the waste-image classification project, we took inspiration from the structure of Lab 2 and built a convolutional neural network (CNN) composed of two convolutional layers.

We trained this initial model for 8 epochs, achieving an accuracy of **64%** on the test set.

Seeking to improve performance, we first applied data augmentation techniques, including rotations, cropping and resizing, and color variations. We then reused the same architecture and extended the training to 15 epochs, reaching **67%** accuracy on the test set.

To further enhance the model, and inspired by the image classification course, we decided to use **pretrained backbones**. To familiarize ourselfs with this approach, we started with **DenseNet**, one of the first models introduced in the course. We then trained it by freezing the entire backbone and training only the classification head, which resulted in **92% accuracy** on the training set.

However, these good results turned out to be misleading: the model achieved only **64%** accuracy on the test set. This large gap between training and test performance revealed a clear case of **overfitting**.

By analyzing the cause, we realized that the issue came from the absence of a separate validation set. Initially, we only had a training set and a test set, which made it difficult to determine the optimal stopping point or adjust hyperparameters.

Even though we had “only” 4,279 images, we concluded that keeping a validation subset was essential. We therefore split the training data, reserving **15%** for validation (as recommended in the first course), while keeping the originally provided test set untouched. This approach allowed us to better assess the model’s generalization capability and avoid overfitting in subsequent iterations.

Optimizing with Backbones and Progressive Fine-Tuning

During our second attempt, we again froze the entire backbone except for the head. However, the validation accuracy quickly plateaued at around **75%**, which was disappointing. This made us realize the importance of a **progressive fine-tuning strategy**: first training the head, then gradually unfreezing deeper layers.

We therefore adopted a **three-phase training strategy**:

Phase 1: Training the Classification Head

We froze all backbone layers and trained only the classification head for 6–7 epochs. We manually stopped training once the validation accuracy stabilized and ensured that the loss was not increasing, which would indicate overfitting.

Phase 2: Partial Unfreezing

Once the head was properly trained, we unfroze the deepest layers of the model (DenseNet’s *denseblock4* and *norm5*). We also reduced the learning rate to avoid disrupting the pretrained weights. After 4–5 additional epochs, the model again reached a stable point.

Phase 3: Full Unfreezing

Finally, we unfroze the entire backbone and continued training. After 5–6 more epochs, the validation accuracy stabilized around **85%**.
Evaluating the model on the test set yielded **86% accuracy**.

Encouraged by these results, we experimented with a more recent backbone from the course: **ConvNeXt**. We applied the same three-phase strategy, progressively unfreezing layers 6 and 7 during the partial-unfreeze stage.

To further optimize training, we added a **checkpointing system** that saved the best model based on validation accuracy. This ensured that even if we trained for too long, we still kept the optimal version.

Using this systematic approach with **ConvNeXt-Tiny** (chosen because the project did not require the most expensive large models), we achieved **95% test accuracy**, a significant improvement over the 86% obtained with DenseNet.

Ensembling Multiple Backbones

To push performance even further, we experimented with combining three backbones with different architectures, so that they would approach the classification task in different ways. We kept **ConvNeXt** (CNN), added **Swin-T** (Vision Transformer), and included **EfficientNetV2-M**, which, although also a CNN, differs significantly from ConvNeXt:

- ConvNeXt uses large convolutions and Transformer-inspired blocks.
- EfficientNet uses MBConv blocks and is optimized through **Compound Scaling**, leading to a very different structure and learning behavior.

We trained Swin-T and EfficientNetV2-M using the same fine-tuning strategy as ConvNeXt. Then we computed a **weighted average** of their predictions based on validation performance. Applying this weighted ensemble to the test set and generating *submission.csv* resulted in **over 97% accuracy**.

This demonstrated the power of weighted ensembling, though we questioned whether the extra cost of maintaining three large backbones was justified to gain only 2% over ConvNeXt alone.

Lightweight Models and Cost–Accuracy Trade-offs

After working with very powerful but heavy models, we wanted to evaluate how much accuracy we could obtain with significantly lighter, cheaper models. This is especially relevant for applications such as **TinyML** or embedded systems, where memory and compute constraints are critical.

We therefore tried **MobileNet**:

- **MobileNet-Small** reached only **79% test accuracy**, which was too low despite its excellent portability.

- **MobileNet-Large**, trained with the same methodology as the other models, reached **94% test accuracy**, only slightly below ConvNeXt's 95%.

The size comparison is striking:

Model	Size
MobileNet-Large	16 MB
ConvNeXt-Tiny	109 MB
EfficientNetV2-M	208 MB
Swin-T	107 MB
Three-model ensemble	424 MB

This means the ensemble was **26× larger than MobileNet-Large** for only a **3% accuracy gain** (97% vs 94%).

Through this project, beyond learning technical concepts such as partial fine-tuning, data augmentation, and weighted ensembling, we also learned the importance of **balancing model cost and accuracy**. In many real-world scenarios—especially embedded systems—it is far more valuable to use a lightweight model that is nearly as accurate as a much heavier and more expensive alternative.

Lastly, in the notebook, we included only the fine-tuning process for EfficientNet, as we applied the same technique to all other models.

Testing on ONNX and TFLITE models

After getting a proper model that got a punctuation of 97%, we have transformed the .pth model into a .onnx model and a .tflite model, in order to test its efficiency. To test how fast the code executes, we used the webpage shared by Mr. Gaetan: http://gaetanbahl.engineer:2022/.

When using the webpage, we have ran each of the models a total of 10 times, getting the following data:

	ONNX	**TFLITE**
Iteration 1	892.572535	1027.748372
Iteration 2	892.481807	1033.102939
Iteration 3	907.132090	1027.483020
Iteration 4	894.016201	1028.047389
Iteration 5	906.890959	1027.457399
Iteration 6	891.880993	1032.912381
Iteration 7	907.682343	1030.249014
Iteration 8	893.721218	1027.438929
Iteration 9	906.778978	1027.338447
Iteration 10	891.615036	1027.428981
Average	898,477216	1028,9206871

Execution speed for the ONNX and TFLITE models (in ms)

As we can see, the ONNX model is always faster than the TFLITE model, even though they are not so far apart. Both of them show a very stable time of execution, not diverging much from the average value.

The ONNX model being faster on this test does not mean that it will always be faster, since both model's execution speed are very dependent on the environment they are being used in. This test just shows that the ONNX model is faster when executing on a benchmark. This makes sense, since the TFLITE models are usually designed to be run on embedded systems.

Experimenting with distillation

Finally, we have tried the distillation technique to try to get a better outcome, where a smaller model, usually called the student, learns from both ground-truths and from a pretrained model, the teacher.

In this experiment, a ConvNeXt-based architecture was employed as the student model. The teacher model provided soft-label predictions, which guided the training of the student through a combination of standard cross-entropy loss on true labels and a distillation loss derived from the teacher's outputs. Training was conducted over 17 epochs using a standard optimization setup.

Even though the program was meant to last for 60 epochs, it stopped at 17 since that was the point where the val_loss value stopped improving. This happened when the val_loss value was 1.0204, and the train_loss value was 0.7009. The accuracy of the model at this point was 75%, very far away from the previous results.

This means that the distillation technique using the ConvNext model is not working as good as the ConvNext model with the backbones. Therefore, we will be sticking with the 97% accuracy previously achieved as the best result.